Primer: Guided Cost Learning (GCL)

Finn, Levine, and Abbeel (2016)

econirl package documentation

1 Context and Problem

Maximum entropy IRL (Ziebart et al. 2008) learns a cost function $c_\theta(\tau)$ that explains expert behavior by maximizing the entropy-regularized likelihood of demonstrated trajectories. The key computational challenge is the partition function $Z = \int \exp(-c_\theta(\tau))\, d\tau$, which sums over all possible trajectories. Early methods computed Z exactly via dynamic programming, but this requires known transition dynamics and a discrete, tractable state space. For continuous-state robotics tasks with unknown dynamics, exact computation of Z is infeasible.

Previous approaches to this problem used linear cost functions $c_\theta = \theta^\top f(x, u)$ with hand-crafted features. This simplifies the cost learning problem but limits expressiveness and requires domain expertise to design the features. Wulfmeier et al. (2016) proposed using neural networks for the cost function but still required known dynamics for the inner MDP solve. Sample-based methods (Boularias et al. 2011, Kalakrishnan et al. 2013) estimated Z with importance sampling from a fixed background distribution, but the fixed distribution produces high-variance estimates when the cost function changes during training.

Finn, Levine, and Abbeel (2016) proposed Guided Cost Learning to solve both problems simultaneously. GCL uses a neural network to parameterize the cost function, removing the need for hand-crafted features. It estimates the partition function via importance sampling from an adaptively learned policy, not a fixed background distribution. The policy is updated by reinforcement learning to track the current cost, producing lower-variance importance weights as training progresses. The result is a model-free IRL algorithm that learns nonlinear cost functions from trajectory demonstrations without requiring known dynamics.

GCL occupies a specific position in the econirl estimator taxonomy. It is a trajectory-level IRL method, operating on full trajectories rather than per-step state-action pairs. MCE-IRL matches per-step feature expectations via occupancy measures, which requires solving the MDP at each gradient step. GCL replaces this with trajectory-level importance sampling, trading the exact occupancy-measure gradient for a sample-based estimate that works without a model. Deep MaxEnt IRL (Wulfmeier 2016) also uses neural costs but relies on exact value iteration, so it requires known transitions. GCL does not.

2 Model and Notation

Symbol	Meaning
$x_t \in \mathcal{X}$, $u_t \in \mathcal{U}$	State, action (continuous or discrete)
$\tau = (x_1, u_1, \ldots, x_T, u_T)$	Trajectory of length T
$c_\theta(x_t, u_t)$	Neural cost function (state-action)
$c_\theta(\tau) = \sum_t c_\theta(x_t, u_t)$	Trajectory cost (sum of per-step costs)
$p(\tau) = \frac{1}{Z}\exp(-c_\theta(\tau))$	MaxEnt trajectory distribution
$Z = \int \exp(-c_\theta(\tau))\, d\tau$	Partition function
$q(\tau)$	Sampling policy (adapted during training)
$w_j = \exp(-c_\theta(\tau_j))/q(\tau_j)$	Importance weight for sample j
$\tilde{w}_j = w_j / \sum_j w_j$	Normalized importance weight
$\mathcal{D}_{\text{demo}} = \{\tau_i\}_{i=1}^N$	Expert demonstrations
$\mathcal{D}_{\text{samp}}$	Sampled trajectories (accumulated)

2.1 Maximum Entropy IOC Objective

The expert is modeled as sampling trajectories from the maximum entropy distribution $p(\tau) \propto \exp(-c_\theta(\tau))$, where lower-cost trajectories are exponentially more likely. The inverse optimal control (IOC) objective is the negative log-likelihood of the demonstrations under this model:

$$\mathcal{L}_{\text{IOC}}(\theta) \;=\; \frac{1}{N}\sum_{i=1}^{N} c_\theta(\tau_i^{\text{demo}}) \;+\; \log Z. \tag{1}$$

The first term pushes the cost of demonstrated trajectories down. The second term (the log-partition function) prevents the cost from collapsing to $-\infty$ everywhere by normalizing.

2.2 Importance-Sampled Gradient

Since Z cannot be computed in closed form for continuous domains, GCL estimates it with importance sampling from a background distribution $q(\tau)$:

$$\frac{\partial \mathcal{L}_{\text{IOC}}}{\partial \theta} \;=\; \frac{1}{N}\sum_{i=1}^{N} \nabla_\theta c_\theta(\tau_i^{\text{demo}}) \;-\; \sum_j \tilde{w}_j\, \nabla_\theta c_\theta(\tau_j^{\text{samp}}), \tag{2}$$

where $\tilde{w}_j \propto \exp(-c_\theta(\tau_j))/q(\tau_j)$ are normalized importance weights. The gradient has an intuitive form: increase the cost of demonstrated trajectories less than the importance-weighted cost of sampled trajectories. At convergence, the cost of demonstrations equals the expected cost under the model, and the gradient vanishes.

3 Identification and Theory

Proposition 1 (Optimal sampling distribution)**.** *The importance sampling variance of the partition function estimate is minimized when $q(\tau) \propto \exp(-c_\theta(\tau))$, the model distribution itself. Since c_θ changes during training, q must be adapted to track the current cost.*

This motivates the "guided" part of GCL: the sampling policy q is updated by solving the forward control problem under the current cost c_θ, driving q toward the optimal importance distribution. In continuous-action domains, this uses trajectory optimization with KL-divergence constraints (Levine and Abbeel 2014). In the tabular DDC setting of econirl, the forward solve is soft value iteration, which is exact for discrete state-action spaces.

Cost ambiguity. Like all MaxEnt IRL methods, GCL identifies the cost function only up to additive constants and potential-based shaping. For any bounded $h(x)$, the shaped cost $c'(x, u) = c(x, u) + \gamma\, \mathbb{E}[h(x')] - h(x)$ produces the same optimal policy under the original dynamics. Unlike AIRL, GCL does not structurally disentangle the reward from the shaping potential, so the recovered cost is not guaranteed to transfer across environments.

Neural cost expressiveness. The neural network parameterization $c_\theta(x, u)$ can represent arbitrary nonlinear cost functions, subject to the network architecture. This is strictly more expressive than the linear specification $c_\theta = \theta^\top f(x, u)$ used by MCE-IRL. The cost is that the recovered cost has no interpretable parameters and does not support standard errors. GCL trades interpretability for flexibility, making it suitable for tasks where the true cost structure is unknown and feature engineering is impractical.

Importance weight variance. The quality of the gradient estimate (2) depends on how close the sampling distribution $q(\tau)$ is to the model distribution $p(\tau) \propto \exp(-c_\theta(\tau))$. When q is far from p, a few trajectories dominate the importance weights and the effective sample size collapses. The guided sampling procedure addresses this by continuously adapting q toward p via policy optimization. Importance weight clipping at $w_{\max}$ (default 10.0 in econirl) provides a safety net: it caps the influence of any single trajectory at the cost of introducing bias. As q approaches p, the clipping becomes inactive and the bias vanishes.

Convergence. Finn et al. (2016) do not provide formal convergence guarantees for the full GCL algorithm. The IOC inner loop converges under standard conditions for stochastic gradient descent on smooth objectives. The outer loop alternates between cost learning and policy optimization, and convergence of this alternating scheme depends on step sizes and the quality of the importance sampling estimates. In practice, training is monitored by the policy's action-prediction accuracy on held-out demonstrations. When accuracy plateaus, the cost function has stabilized and further iterations provide diminishing returns.

Regularization. The paper introduces a gradient penalty $\mathcal{R}_1 = \lambda_1 \mathbb{E}[\|\nabla_x c_\theta\|^2]$ that encourages smooth cost landscapes, plus a monotonicity hinge loss for episodic tasks. In econirl, importance weight clipping provides sufficient regularization for tabular DDC problems.

4 Estimation Algorithm

Algorithm 1: Guided Cost Learning (Finn, Levine, and Abbeel 2016)

Input: Demonstrations $\mathcal{D}_{\text{demo}} = \{\tau_1, \ldots, \tau_N\}$, transitions P, discount β
Initialize neural cost c_θ with random weights `// _create_cost_function()`
Initialize policy π (uniform or from demonstrations)
$\mathcal{D}_{\text{samp}} \leftarrow \emptyset$

for *iteration* $i = 1, \ldots, I$ **do**

Step 1: Sample trajectories `// _sample_trajectories()`
Sample M trajectories $\{\tau_j\}$ from current policy π
$\mathcal{D}_{\text{samp}} \leftarrow \mathcal{D}_{\text{samp}} \cup \{\tau_j\}$

Step 2: Compute importance weights `// _compute_importance_weights()`
for *each* $\tau_j \in \mathcal{D}_{samp}$ **do**
$w_j \leftarrow \exp(-c_\theta(\tau_j)) \,/\, q(\tau_j)$ `// Eq. 2`
end
Clip: $w_j \leftarrow \min(w_j,\, w_{\max})$
Normalize: $\tilde{w}_j \leftarrow w_j / \sum_j w_j$

Step 3: Update cost network
$g \leftarrow \frac{1}{N} \sum_i \nabla_\theta c_\theta(\tau_i^{\text{demo}}) - \sum_j \tilde{w}_j \, \nabla_\theta c_\theta(\tau_j^{\text{samp}})$ `// Eq. 2`
$\theta \leftarrow \theta - \eta \, g$ (Adam optimizer)

Step 4: Update policy `// _update_policy()`
Compute reward $R(s, a) = -c_\theta(s, a)$
Solve soft Bellman for V, π via hybrid value iteration

end

Output: Cost function c_θ, policy π, reward table $R(s, a) = -c_\theta(s, a)$

Code annotations reference methods in `src/econirl/contrib/gcl.py`. The cost network uses learned embeddings of dimension 32 for states and actions, concatenated and passed through a two-layer MLP with ReLU activations (`NeuralCostFunction` in the preferences module). Default configuration: 200 outer iterations, 100 sampled trajectories per iteration, importance weight clipping at 10.0, Adam learning rate 10^{-3}, hybrid inner solver with tolerance 10^{-8}.

5 Simulation

The companion script `gcl_demo.py` runs GCL alongside MCE-IRL on a 5×5 gridworld (25 states, 5 actions, $\beta = 0.99$, 10,000 observations).

```
from econirl.contrib.gcl import GCLEstimator, GCLConfig

config = GCLConfig(hidden_dims=[32, 32], cost_lr=0.001,
    max_iterations=100, n_sample_trajectories=50)
result = GCLEstimator(config=config).estimate(
    panel, utility, problem, transitions)
```

Table 1: GCL vs MCE-IRL on 5×5 gridworld ($\beta = 0.99$, 10,000 obs).

	MCE-IRL	GCL
Policy accuracy	100.0%	84.0%
Time (seconds)	83.66	99.35

On this linear-reward gridworld, MCE-IRL achieves 100 percent policy accuracy because the true reward is linear in the features and MCE-IRL's model is correctly specified. GCL achieves 84 percent, lower because it learns a nonparametric neural cost from trajectory-level importance sampling, which is a harder estimation problem than matching per-step feature expectations. GCL does not benefit from knowing the correct features and must discover the cost structure from raw state-action pairs.

When GCL wins. GCL's advantage appears on problems where the true cost is nonlinear and the correct features are unknown. On Objectworld and Binaryworld environments, where the reward depends nonlinearly on distances to colored objects or on count thresholds over binary neighborhoods, MCE-IRL with linear features cannot capture the true cost structure while GCL's neural network can. The trajectory-level importance sampling also handles unknown dynamics gracefully: GCL never needs the transition matrix $P(s'|s, a)$ during training if the sampling policy generates trajectories by interacting with the environment directly. In the econirl tabular setting, soft value iteration is used for the policy update, which does require P, but this is a design choice for computational convenience rather than a fundamental requirement of the method.

Comparison to Deep MaxEnt. Both GCL and Deep MaxEnt IRL use neural cost functions, but they differ in how they estimate the partition function gradient. Deep MaxEnt computes the occupancy measure exactly via the forward pass of value iteration, which requires known transitions and a tractable state space. GCL estimates the same gradient via importance-sampled trajectories, which works with unknown dynamics but introduces sampling variance. On tabular problems where value iteration is cheap, Deep MaxEnt is more precise. On continuous-state problems where value iteration is infeasible, GCL is the only option among the two.

Comparison to AIRL. AIRL also learns from trajectories without known dynamics, but it structurally disentangles the reward from the value function via $f(s, a, s') = g(s) + \gamma h(s') - h(s)$, guaranteeing that $g(s)$ recovers the true state-only reward up to a constant. GCL makes no such separation, so its cost does not transfer across environments. However, GCL's cost is state-action dependent $c(x, u)$, while AIRL's guarantee requires state-only rewards, making GCL the more natural specification for DDC models where different actions have different payoffs.

Practical guidance. Use GCL when the true cost function is nonlinear, the correct features are unknown, and interpretability is not required. If the cost is linear in known features, MCE-IRL is strictly better because it estimates fewer parameters and produces standard errors. If transferable rewards are needed, use AIRL. If the state space is discrete and small, Deep MaxEnt IRL is more precise because it avoids the importance sampling variance. GCL is most valuable in continuous-state, continuous-action settings where neither exact value iteration nor feature engineering is feasible. The companion script `gcl_demo.py` demonstrates the full workflow. Full documentation is at `docs/estimators/gcl.md`.